

Использование блокчейн-технологий для построения системы электронных голосований

Выпускная квалификационная работа

Выполнил: Бактыбеков Нурсултан Бактыбекович, гр. WIN-1-22

Научный руководитель: Самсалиев А.А., к.т.н., доцент

01.04.2026

Кыргызско-Германский институт прикладной информатики

Направление: «Информатика», профиль: «Веб Информатика»

Проблемы традиционных систем электронного голосования:

- **Единая точка отказа (Single Point of Failure)** — центральный сервер уязвим к DDoS-атакам и техническим сбоям
- **Угроза фальсификаций (Data Tampering)** — администраторы БД технически способны незаметно изменять записи голосов
- **Отсутствие верифицируемости** — процесс подсчёта закрыт от наблюдателей («чёрный ящик»)
- **Проблема доверия** — избиратель не может убедиться, что его голос учтён корректно

Вывод: необходима система, где результаты математически невозможно подделать, а процесс подсчёта открыт для всех.

Цель: Разработать децентрализованную систему электронного голосования на базе смарт-контрактов блокчейна **Solana**, обеспечивающую прозрачность, криптографическую неизменность результатов и защиту от двойного голосования.

Задачи:

1. Провести анализ предметной области и недостатков существующих подходов
2. Проанализировать существующие блокчейн-платформы и обосновать выбор Solana
3. Разработать архитектуру системы и спроектировать модели данных (структуры счетов)
4. Реализовать смарт-контракт на языке **Rust** с применением фреймворка **Anchor**
5. Покрыть логику автоматизированными интеграционными тестами на **TypeScript**

Анализ существующих блокчейн-платформ для e-voting

Платформа	Преимущества	Недостатки
Ethereum / форки	Первая платформа смарт-контрактов, широкое сообщество	15–30 tx/sec, газовые комиссии \$10–50 за транзакцию
Hyperledger / Exonum	Высокая скорость, бесплатные транзакции	Приватная сеть — узлами владеют организаторы. Нарушает принцип децентрализации
Solana ✓	До 65 000 tx/sec, комиссия \$0.00025, публичная Permissionless-сеть	Сложность разработки смарт-контрактов (Rust)

Механизм консенсуса Solana: Proof-of-History

Solana использует уникальный гибридный механизм **PoH + PoS**:

- **Proof-of-History (PoH)**: криптографически верифицированные временные метки позволяют каждому узлу независимо и последовательно упорядочивать транзакции без дополнительной координации
- **Proof-of-Stake (PoS)**: валидаторы вносят залог SOL и голосуют за корректность состояния реестра

Ключевые характеристики для системы голосования:

- Пропускная способность: до **65 000 транзакций/сек**
- Стоимость одного голоса: **≈ 0.00025 USD**
- Финальность транзакции: **400 мс**
- Публичный Permissionless-доступ: любой может стать валидатором и проверить реестр

Акторы системы:

1. Инициатор (Admin / Owner)

- Создает предложение для голосования через инструкцию `create(pid, description)`
- Оплачивает rent-fee за выделение on-chain памяти под счёт Proposal
- Параметры: уникальный числовой ID (u8) и описание длиной до 50 байт

2. Избиратель (Voter)

- Идентифицируется через криптографический Solana-кошелек (`Signer<'info>`)
- Вызывает `vote_yes()` или `vote_no()` для активного предложения
- Не может проголосовать дважды: это гарантируется архитектурой PDA на уровне среды исполнения блокчейна

Архитектура децентрализованного приложения (dApp)

On-Chain слой (блокчейн Solana):

- Смарт-контракт `e_voting` (Rust + Anchor) — ID: `7UfykF9i...`
- Хранит состояние в двух типах счетов: `Proposal` и `Vote`
- Инструкции: `create`, `vote_yes`, `vote_no`
- Авторизация и проверка подписей — встроено в runtime Solana

Off-Chain слой (клиент):

- Кошелёк пользователя (Phantom, CLI Solana)
- RPC-вызовы к кластеру через `@coral-hyz/anchor`
- Формирование и отправка транзакций
- Тестовая среда: `solana-test-validator` (Localnet)

Поток данных: Кошелёк → подписанная транзакция → RPC-узел → консенсус валидаторов → запись в реестр счетов

Модель данных: структура Proposal

Хранит состояние голосования. Адрес счёта вычисляется детерминированно по seeds:
["proposal", owner_pubkey, pid]

Поле	Тип	Описание
pid	u8	Числовой идентификатор предложения
description	String	Описание — строго до 50 байт (<code>#[max_len(50)]</code>)
yes_votes	u32	Счётчик голосов «За»
no_votes	u32	Счётчик голосов «Против»
ongoing	bool	Флаг активности голосования
owner	Pubkey	Публичный ключ организатора
bump	u8	Криптографическое смещение (nonce) PDA

Модель данных: структура Vote

Фиксирует факт голосования конкретного пользователя. Адрес: ["vote", proposal_pubkey, user_pubkey]

Поле	Тип	Описание
user	Pubkey	Публичный ключ кошелька избирателя
proposal	Pubkey	Адрес счёта предложения, за которое отдан голос
vote	VoteType	Тип голоса: YesVote или NoVote
bump	u8	Nonce для вычисления PDA

Защита от двойного голосования: user_pubkey является частью seed. Второй счёт Vote с тем же user и proposal вычислился бы по тому же адресу — runtime Solana вернёт ошибку AlreadyInUse.

Требования к системе и критерии оценки качества

Функциональные требования:

- Создание голосования с уникальным ID и описанием
- Голосование «За» / «Против» с подтверждением подписью кошелька
- Архитектурный запрет двойного голосования (PDA-механизм)
- Публичный доступ к результатам в режиме реального времени

Нефункциональные (качественные) критерии:

Критерий	Реализация
Отказоустойчивость	100% — делегирована консенсусу Solana
Неизменность данных	Криптографическая цепочка блоков (Immutability)
Безопасность памяти	Гарантирована borrow-checker языка Rust
Верифицируемость	Открытый реестр, любой может пересчитать результат

- 1. Цифровая подпись транзакций (Ed25519):** Каждый голос подписывается закрытым ключом кошелька. Solana runtime верифицирует подпись до исполнения инструкции: `pub user: Signer<'info>`. Подделать транзакцию без приватного ключа невозможно.
- 2. Program Derived Addresses (PDA):** Алгоритм: `SHA256(seeds || program_id)` итерируется с уменьшающимся bump до нахождения точки вне кривой Ed25519. Смарт-контракт является единственным, кто может подписывать от имени PDA. Это исключает несанкционированное создание счётов сторонними программами.
- 3. Безопасная арифметика:** Инкремент счётчиков через `checked_add(1).ok_or(EVotingError::MaxYesVotesReached)?` предотвращает целочисленное переполнение (u32 overflow) — критическую уязвимость в финансово-чувствительных программах.

Реализация: создание голосования (create)

```
pub fn create(ctx: Context<CreateProposal>, pid: u8, description: String)
-> Result<()> {
    require!(
        description.as_bytes().len() <=
Proposal::DESCRIPTION_MAXIMUM_LENGTH,
        EVotingError::DescriptionTooLong // возвращает ошибку при len >
50
    );
    let proposal = &mut ctx.accounts.proposal;
    proposal.pid      = pid;
    proposal.description = description;
    proposal.yes_votes  = 0;
    proposal.no_votes   = 0;
    proposal.ongoing     = true;
    proposal.owner       = *ctx.accounts.user.key;
```

Реализация: создание голосования (create)

```
proposal.bump      = ctx.bumps.proposal;  
Ok()  
}
```

Атрибут `init` в контексте `CreateProposal` автоматически выделяет `8 + Proposal::INIT_SPACE` байт и инициализирует счёт с дискриминатором `Anchor`.

Реализация: алгоритм голосования (vote_yes / vote_no)

```
fn vote(ctx: Context<AddVote>, vote_type: VoteType) -> Result<()> {  
    let proposal = &mut ctx.accounts.proposal;  
    let vote      = &mut ctx.accounts.vote;  
  
    require!(proposal.ongoing == true,  
EVotingError::VotingSessionIsClosed);  
  
    vote.user      = *ctx.accounts.user.key;  
    vote.proposal  = proposal.key();  
    vote.bump      = ctx.bumps.vote;  
  
    match vote_type {  
        VoteType::YesVote => {  
            vote.vote      = VoteType::YesVote;  
            proposal.yes_votes = proposal.yes_votes
```

Реализация: алгоритм голосования (vote_yes / vote_no)

```
        .checked_add(1).ok_or(EVotingError::MaxYesVotesReached)?;
    }
    VoteType::NoVote => {
        vote.vote      = VoteType::NoVote;
        proposal.no_votes = proposal.no_votes
            .checked_add(1).ok_or(EVotingError::MaxNoVotesReached)?;
    }
}
Ok(())
}
```

Результаты интеграционного тестирования

Тесты запускаются через `anchor test` на локальном кластере (`solana-test-validator`).

Тест	Описание	Итог
Описание > 50 байт	<code>create(0, "Very large description...")</code> → ошибка <code>DescriptionTooLong</code>	✓ Пройден
Создание Proposal #1	<code>create(1, "Proposal #1")</code> → <code>yes_votes: 0, no_votes: 0, ongoing: true</code>	✓ Пройден
Создание Proposal #2	<code>create(2, "Proposal #2")</code> → аналогичная инициализация	✓ Пройден
Джо голосует «За» #1	<code>vote_yes()</code> → <code>yes_votes: 1, no_votes: 0</code>	✓ Пройден
Juliana голосует «Против» #1	<code>vote_no()</code> → <code>yes_votes: 1, no_votes: 1</code>	✓ Пройден
Neto голосует «Против» #1	<code>vote_no()</code> → <code>yes_votes: 1, no_votes: 2</code>	✓ Пройден

Результаты интеграционного тестирования

```
> ls
[t] Anchor.toml  Cargo.toml  node_modules  programs  target  tsconfig.json
Cargo.lock      migrations  package.json  README.md  tests   yarn.lock
> anchor test
WARNING: `anchor-lang` version(0.30.0) and the current CLI version(0.32.1) don't match.

    This can lead to unwanted behavior. To use the same CLI version, add:

    [toolchain]
    anchor_version = "0.30.0"

    to Anchor.toml

WARNING: `@coral-xyz/anchor` version(^0.30.0) and the current CLI version(0.32.1) don't match.

    This can lead to unwanted behavior. To fix, upgrade the package by running:

    yarn upgrade @coral-xyz/anchor@0.32.1

warning: unexpected `cfg` condition value: `custom-heap`
  -> programs/e-voting/src/lib.rs:5:1
  |
5 | #[program]
  | ^^^^^^^^^^
  |
  = note: expected values for `feature` are: `cpi`, `default`, `idl-build`, `no-entrypoint`, `no-idl`, and `no-log-ix-name`
  = note: using a cfg inside a macro will use the cfgs from the destination crate and not the ones from the defining crate
  = help: try referring to `$crate::custom_heap_default` crate for guidance on how handle this unexpected cfg
  = help: the macro `$crate::custom_heap_default` may come from an old version of the `solana_program` crate, try updating yo
ur dependency with `cargo update -p solana_program`
  = note: see <https://doc.rust-lang.org/nightly/rustc/check-cfg/cargo-specifics.html> for more information about checking co
nditional configuration
  = note: `#[warn(unexpected_cfgs)]` on by default
```

Рис. 1. Запуск тестов

Результаты интеграционного тестирования

```
Finished `release` profile [optimized] target(s) in 0.13s
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.10s
Running unittests src/lib.rs (/home/orenvadi/Desktop/vkr/code/e-voting/target/debug/deps/e_voting-50ba037dd304387f)

Found a 'test' script in the Anchor.toml. Running it as a test suite!

Running test suite: "/home/orenvadi/Desktop/vkr/code/e-voting/Anchor.toml"

yarn run v1.22.22
warning package.json: No license field
$ /home/orenvadi/Desktop/vkr/code/e-voting/node_modules/.bin/ts-mocha -p ./tsconfig.json -t 1000000 'tests/**/*.ts'
(node:70270) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///home/orenvadi/Desktop/vkr/code/e-voting/tests/e-voting.ts is not specified and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to /home/orenvadi/Desktop/vkr/code/e-voting/package.json.
(Use `node --trace-warnings ...` to show where the warning was created)

e-voting system
  ✓ Proposal cannot have large description (greater than 50)
  ✓ Proposal creation #1 (408ms)
  ✓ Proposal creation #2 (416ms)
  ✓ Joe is voting yes (proposal #1) (417ms)
  ✓ Juliana is voting no (proposal #1) (416ms)
  ✓ Neto is voting no (proposal #1) (416ms)

6 passing (4s)

Done in 4.54s.
```

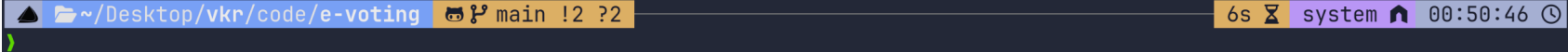


Рис. 2. Запуск тестов

Спасибо за внимание!

Буду рад ответить на ваши вопросы.